

Giải phẫu một hệ thống thị giác (Anatomy of a Vision System)

PHỤ THUỘC không bắt buộc; đây là chương mở đầu của Phần B. Đọc trước A/02-vat-ly-tao-anh nếu muốn hiểu vì sao “luồng dữ liệu” phải bắt đầu từ cảm biến chứ không từ tensor đầu vào.

TÓM TẮT

Chương này đưa ra ba thứ. Một quy trình bảy câu hỏi để mổ xẻ một repo hoặc paper lạ. Một bảng nói rõ paper, code và **ablation** mỗi nguồn che giấu điều gì. Và nguyên lý *độ khó không biến mất, chỉ bị đẩy đi chỗ khác*, công cụ dùng được suốt phần còn lại của cây. Đọc xong, chỉ sau một buổi bạn nói ra được các **chế độ hỏng** (*failure mode*) của một hệ thống mới gặp, thay vì chỉ liệt kê module của nó. Bạn cũng biết lúc nào nên đóng một paper lại thay vì đọc tiếp. Nếu chỉ nhớ một điều: đọc một hệ thống là đi tìm chỗ nó phải trả giá. Ràng buộc cứng loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng, nên hãy tìm ràng buộc trước và tìm ý tưởng sau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận ảnh làm đầu vào và trả về dự đoán (nhãn, hộp, mask). “Huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất; “suy luận” (*inference*) là chạy hàm đó trên dữ liệu mới.
- **hệ thống thị giác và pipeline** — một hệ thống thật không chỉ có mô hình. Nó là một chuỗi khối nối tiếp — cảm biến, tiền xử lý, mô hình, hậu xử lý, quyết định — và chuỗi đó gọi là *pipeline*. Mỗi khối trong chuỗi gọi là một *module*. Phân biệt hai thứ này là điều kiện để đọc chương.
- **tín hiệu giám sát (supervision signal)** — thứ nói cho thuật toán biết dự đoán đúng là gì. Thường là nhãn do người gán nhãn tạo ra, nhưng cũng có thể là một ràng buộc vật lý, một cảm biến khác, hoặc chính dữ liệu (tự giám sát). Biết tín hiệu giám sát đến từ đâu là biết hệ thống học được điều gì và không học được điều gì.
- **giả định (assumption)** — một mệnh đề về thế giới mà hệ thống coi là đúng nhưng không kiểm tra — ví dụ “vật thể luôn nằm trên mặt đất” hay “ảnh đủ sáng”. Giả định không được nói ra vẫn có hiệu lực; hệ thống hỏng đúng lúc giả định bị vi phạm.
- **ablation** — thí nghiệm bỏ đi từng thành phần của hệ thống rồi đo lại kết quả, để biết thành phần nào thực sự đóng góp. Từ này không có bản dịch tiếng Việt thông dụng nên tài liệu giữ nguyên.
- **baseline** — phương pháp đối chứng đơn giản hơn mà phương pháp mới phải vượt qua thì mới có ý nghĩa. Nếu baseline không được tinh chỉnh cẩn thận thì mọi so sánh với nó đều vô giá trị.
- **độ trễ (latency) và FPS** — độ trễ là thời gian xử lý một khung hình, tính bằng mili-giây; FPS (*frames per second*) là số khung hình xử lý được mỗi giây. Hai đại lượng nghịch đảo nhau và là ràng buộc cứng phổ biến nhất trong hệ thống thời gian thực.
- **mAP** — thước đo phổ biến cho bài toán phát hiện vật thể, gộp độ chính xác và độ bao phủ trên nhiều ngưỡng thành một con số duy nhất. Ở chương này bạn chỉ cần biết nó là “điểm số trong bảng kết quả”; định nghĩa đầy đủ nằm ở Phần E.

CÂU HỎI MỞ ĐẦU

- Q1. Hai kỹ sư đọc cùng một repo trong cùng một buổi: vì sao một người ra về với danh sách module, còn người kia dự đoán được hệ thống sẽ sập trong một hành lang tường trắng?
- Q2. Nếu phần method là phần dài nhất của mọi paper, vì sao nó lại là phần đáng đọc *sau cùng*?
- Q3. Một paper báo cáo 45 ms còn bạn có 33 ms mỗi khung hình — vì sao phép tính hai dòng đó loại bỏ nhiều phương án hơn cả bảng kết quả?
- Q4. Nếu bảng ablation nói bỏ khối A không mất gì và bỏ khối B cũng không mất gì, bạn có được kết luận cả hai đều vô dụng không?

1.1 Đặt vấn đề

Đứng trước một repo hoặc một paper lạ, làm sao đi từ “không hiểu gì” tới “biết nó giả định gì và hỏng ở đâu” trong một buổi, thay vì mò mẫm hàng tuần? Câu hỏi này nghe như một câu hỏi về kỹ năng đọc, nhưng thực ra nó là câu hỏi về **cách phân loại**. Người mới đọc hệ thống theo đặc điểm bề mặt: tên module, tên tầng, con số trong bảng kết quả. Người có kinh nghiệm đọc theo cấu trúc sâu: bài toán gì, giả định gì, đánh đổi gì. Hai cách đọc cho ra hai bản đồ hoàn toàn khác nhau về cùng một hệ thống, và chỉ bản đồ thứ hai dùng được khi bạn phải sửa hoặc thay một khối.

Sự khác biệt này không phải chuyện học thuật. Bạn đã gặp nó ở phía SLAM. Hai người cùng đọc ORB-SLAM3 [1] trong một buổi. Người thứ nhất ra về với một danh sách bộ phận: có ORB extractor, có DBoW2, có **hiệu chỉnh chùm tia** (*bundle adjustment*) cục bộ, có **đóng vòng** (*loop closing*). Người thứ hai ra về với một câu đánh cược: hệ này tin rằng góc ảnh còn lặp lại được giữa hai khung hình liên tiếp. Từ câu đó suy ra ngay hai chỗ sập. Ảnh mờ làm góc nhoe đi nên không còn khớp được; bề mặt trơn thì ngay từ đầu đã không có góc nào để tìm. Người thứ hai cũng biết hệ bù lại bằng một tầng **tái định vị** (*relocalization*) đắt tiền. Chỉ anh ta dự đoán được chuyện gì xảy ra trong một hành lang tường trắng. Danh sách bộ phận không sinh ra dự đoán; bản đồ giả định thì có.

Cái làm cho việc đọc trở nên khó là không gian thiết kế quá rộng. Một hệ thống thị giác hiện đại có hàng chục lựa chọn gắn kết nhau, và paper hầu như luôn kể câu chuyện theo trục “chúng tôi nghĩ ra ý tưởng đẹp này”. Trục đó gần như không bao giờ là trục thật đã sinh ra thiết kế. Trục thật thường là một ràng buộc: chỉ có bằng ấy nhân, chỉ có bằng ấy mili-giây, chỉ có bằng ấy bộ nhớ. Vì vậy chương này không dạy bạn đọc nhanh hơn; nó dạy bạn đọc theo đúng trục, để hầu hết nội dung tự rơi vào chỗ của nó.

TRỰC GIÁC

Đọc một hệ thống thị giác giống khám một bệnh nhân chứ không giống đọc một cuốn catalogue. Catalogue liệt kê bộ phận; khám thì hỏi *bộ phận này gánh chức năng gì, và nếu nó hỏng thì triệu chứng gì hiện ra ở đâu*. Một người thuộc lòng tên hai trăm chi tiết máy vẫn không sửa được xe; người biết “nếu bơm xăng yếu thì máy chết ở vòng tua cao” thì sửa được. Thước đo của cả chương là bạn có phát biểu được các triệu chứng hay không.

1.2 Bảy câu hỏi để mổ xẻ (Seven Questions for Dissection)

A. Vì sao thứ tự của các câu hỏi là thông tin

Thứ tự dưới đây không tùy tiện: mỗi câu hỏi chỉ trả lời được khi câu trước đã có đáp án, và mỗi câu sinh ra vì câu trước để lại một chỗ hở. Nói gọn thì mạch đi từ *bài toán* (câu 1–2) sang *không gian thiết kế bị ràng buộc* (câu 3–4) rồi sang *đóng góp thật và chỗ hỏng* (câu 5–7). Đọc lệch thứ tự thì bạn dành công sức cho những chi tiết mà ràng buộc cứng đàng nào cũng xoá bỏ.

Mạch câu hỏi khi mổ xẻ một hệ thống lạ

- 1 Vấn đề** không biết bắt đầu từ đâu, nên bắt đầu từ chỗ dễ nhìn nhất là code.

Giải pháp tách ba tầng **bài toán** → **nguyên lý** → **cài đặt**, và trước khi đọc một dòng code, ép mình viết ra đúng một câu cho mỗi tầng.

Hệ quả nó ngăn bạn nhầm chi tiết framework với ý tưởng cốt lõi. Cách họ viết **data-loader** hay đặt tên **checkpoint** là chi tiết. Vì sao **tín hiệu giám sát** (*supervision signal*) đến từ nguồn này chứ không nguồn khác mới là ý tưởng.

Giá phải trả mất nửa giờ viết ba câu trông tầm thường; đổi lại không mất ba ngày đọc nhầm tầng.
- 2 Vấn đề** “bài toán” vẫn là một khái niệm mơ hồ, ai cũng gật đầu nhưng mỗi người hiểu một kiểu.

Giải pháp định nghĩa nó bằng đúng ba thứ — đầu vào, đầu ra, và tín hiệu giám sát. Mọi thứ còn lại là lựa chọn triển khai.

Giả định ba thứ này xác định bài toán đủ chặt để hai người khác nhau viết ra cùng một bản mô tả.

Hệ quả nếu chưa trả lời được cả ba thì đọc tiếp là vô ích; hãy quay về phần dataset, vì câu trả lời nằm ở đó chứ không ở phần method.
- 3 Vấn đề** không gian thiết kế quá rộng để suy luận bằng trực giác.

Giải pháp truy tìm **ràng buộc cứng** (*hard constraint*) trước tiên: **độ trễ** (*latency*), bộ nhớ, lượng nhân có được, quyền riêng tư, năng lượng.

Giả định tác giả không tự do như phần method làm ra vẻ.

Hệ quả ràng buộc cứng loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng, nên nó giải thích nhiều lựa chọn thiết kế hơn toàn bộ phần method cộng lại. Con số “khoảng 90%” hay được nhắc là kinh nghiệm thực hành chứ không phải kết quả đo được; nhưng chiều của nhận định thì rất vững.
- 4 Vấn đề** thiết kế nào cũng trông đẹp khi đọc trong paper.

Giải pháp áp nguyên lý **độ khó không biến mất, chỉ bị đẩy đi chỗ khác** và hỏi thẳng: họ đã đẩy nó đi đâu — sang dữ liệu, sang hậu xử lý, sang thời gian huấn luyện, hay sang một giả định về đầu vào?

Giả định mọi hệ thống đều trả giá ở đâu đó cho cái nó làm dễ đi ở chỗ khác.

Hệ quả câu trả lời gần như luôn tồn tại; nếu bạn không tìm ra thì kết luận đúng là bạn chưa hiểu hệ thống.
- 5 Vấn đề** không biết phần nào trong bài là đóng góp thật.

Giải pháp đọc bảng **ablation** theo chiều ngược — bỏ thành phần nào thì hệ thống sập, chứ không phải thêm gì thì con số nhích lên.

Giả định ablation được chạy đàng hoàng và **baseline** được tinh chỉnh ngang với phương pháp đề xuất.

Vấn đề mới giả định đó thường sai; có cả một dòng công trình cho thấy phần lớn “tiến bộ” trong một số lĩnh vực biến mất khi baseline được tinh chỉnh tử tế [2], [3]. Chương 24 dạy cách kiểm tra chính giả định này.

↪ **tiền đề** E/24 — độ tin cậy của một bảng ablation là điều kiện để quy trình đọc này còn giá trị

6 Vấn đề

Giải pháp

Hệ quả

paper hầu như không nói về chỗ nó hỏng.

đọc phần *limitations* và *failure case* trước phần kết quả, vì đó là chỗ tác giả không có động cơ phóng đại.

bạn bước vào phần kết quả với một danh sách nghi vấn sẵn có, nên các con số đẹp không còn quyền định khung suy nghĩ của bạn. Nguyên tắc phía sau: hiểu một hệ thống nghĩa là biết nó hỏng ở đâu, không phải biết nó chạy được gì.

7 Vấn đề

Giải pháp

Hệ quả

lỗi thật khi triển khai thường không nằm trong model.

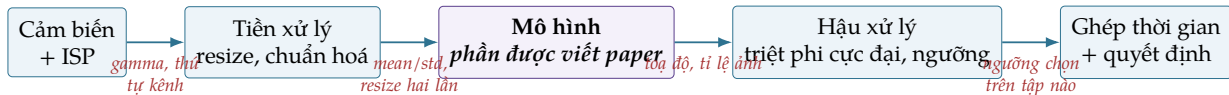
vẽ **sơ đồ luồng dữ liệu** (*data flow diagram*) đầy đủ, từ cảm biến tới quyết định cuối, và đánh dấu mọi đoạn nối giữa hai khối.

phần lớn lỗi hệ thống nằm ở các đoạn nối — sai thứ tự kênh màu, resize hai lần, đơn vị góc lệch nhau, *timestamp* lệch pha — chứ không nằm ở khối được viết paper [4]. Bất kỳ ai từng debug một pipeline SLAM đều đã sống qua điều này.

↪ **tiền đề** A/02 — hai khối đầu của luồng — cảm biến và ISP — là vật lý tạo ảnh, và chúng quyết định phần lớn khoảng cách train/production

B. Luồng dữ liệu: chỗ paper im lặng

Câu hỏi thứ bảy đáng được vẽ ra chứ không chỉ nói suông, vì bản chất của nó là không gian: cái ta muốn thấy là tỉ lệ giữa phần được viết paper và phần còn lại của đường đi. Hình 1.1 cho thấy một luồng điển hình; khối tô đậm là khối duy nhất paper mô tả, và bốn mũi tên nối là nơi các lỗi tồn thời gian nhất sinh ra.



Hình 1.1. Luồng dữ liệu đầy đủ của một hệ thống thị giác. Paper mô tả một khối; các nhãn đỏ dưới mũi tên là những đoạn nối nơi lỗi triển khai thực tế hay sinh ra và không bao giờ xuất hiện trong bảng kết quả.

1.3 Ba nguồn thông tin: paper, code, ablation

Có đúng ba nguồn thông tin về một hệ thống lạ. Người mới thường chọn một nguồn theo thói quen — kỹ sư đọc code, sinh viên đọc paper — rồi tin rằng mình đã hiểu. Vấn đề là mỗi nguồn không chỉ *thiếu* thông tin, nó còn *che giấu* một loại thông tin cụ thể, và loại bị che luôn là loại bạn cần khi phải ra quyết định.

Bảng 1.1. Ba cách tiếp cận một hệ thống lạ, và cái mà mỗi cách che giấu. Cột “che giấu” là cột có giá trị nhất: nó nói cho bạn biết mình đang mù ở đâu.

Cách đọc	Cho bạn biết	Che giấu	Hỏng khi nào
Đọc paper	Ý định của tác giả, vị trí lịch sử của ý tưởng	Chi tiết thật sự quyết định kết quả: gán mẫu (<i>label assignment</i>), augmentation, lịch học	Khi câu chuyện được viết lại sau khi kết quả đã có
Đọc code	Sự thật về cái đang chạy, kể cả chỗ paper không kể	Vì sao chọn như vậy; đâu là thiết yếu, đâu là tàn dư lịch sử	Khi repo lớn tới mức bạn hiểu chi tiết mà mất tầng bài toán
Đọc ablation	Trọng số đóng góp thật của từng khối theo chính tác giả	Tương tác giữa các khối, chất lượng tinh chỉnh của baseline	Khi hai khối làm cùng một việc và che lẫn nhau

Bảng 1.1 cho thấy không nguồn nào đủ một mình, và quan trọng hơn, cho thấy cách phối hợp hiệu quả nhất: đọc ablation trước để biết phần nào đáng quan tâm, rồi đọc code của đúng phần đó, rồi mới quay lại paper để hiểu tác giả nghĩ gì — ngược hoàn toàn với thứ tự trang giấy. Lý do là ablation có tỉ lệ tín hiệu trên độ dài cao nhất, còn phần method có tỉ lệ thấp nhất vì nó được viết để thuyết phục người phản biện. Đây là dạng đọc chọn lọc mà Keshav mô tả dưới tên “đọc ba lượt” [5], chỉ khác là ta chọn điểm vào theo ablation thay vì theo phần mở đầu.

Cột cuối của bảng đáng được nói rõ hơn ở một điểm: ablation cho biết *đóng góp biên* của một khối khi các khối còn lại đã có mặt, chứ không cho biết giá trị tuyệt đối của nó. Hai khối làm cùng một việc sẽ che nhau — bỏ khối nào đi một mình cũng không mất gì, nên bảng kết luận cả hai đều vô dụng, trong khi bỏ cả hai thì hệ thống sập. Chuyện này xảy ra thường xuyên và giải thích vì sao ablation không thay được việc đọc code.

1.4 Độ khó không biến mất (Conservation of Difficulty)

A. Nguyên lý và ba ví dụ

Không có thiết kế nào làm cho một bài toán dễ đi; chỉ có thiết kế di chuyển phần khó sang một trục mà bạn sẵn lòng trả giá hơn. Nói gọn thì mỗi “cải tiến” là một phép đổi tiền tệ, và việc của bạn khi đọc là tìm ra tỉ giá.

• Ba chỗ độ khó hay bị đẩy tới, kèm ví dụ cụ thể:

- **Sang thời gian huấn luyện.** Mô hình *one-stage* bỏ bước sinh đề xuất vùng nên chạy nhanh hơn. Nhưng vẫn phải quyết định vị trí nào trên ảnh chịu trách nhiệm cho vật nào. Câu hỏi đó không biến mất, nó chỉ đổi tên thành bài toán gán mẫu và kéo theo cả một dòng heuristic [6], [7]. DETR bỏ được **triệt phi cực đại** (*non-maximum suppression*) và **anchor**, đổi lại số epoch cần để hội tụ tăng vọt [8].
- **Sang giả định về đầu vào.** SLAM trực tiếp bỏ được bước trích đặc trưng, đổi lại phải giả định **độ sáng không đổi** (*brightness constancy*) giữa các khung hình và chấp nhận một miền hội tụ hẹp hơn nhiều. Đây chính là trục *giả định mạnh với ít dữ liệu* đổi lại *giả định yếu với nhiều dữ liệu* mà chương 4 sẽ đặt tên.

→ cùng nguyên lý với B/04 — “độ khó không biến mất là phát biểu định tính của cùng một trục mà chương 4 định lượng

- **Sang dữ liệu và hậu xử lý.** Một hệ thống tự nhận là “đơn giản, không cần module X” thường đã chuyển gánh nặng sang chỗ khác. Chỗ đó có thể là một tập dữ liệu lớn hơn

nhiều lần. Cũng có thể là một chuỗi luật lọc đầu ra chiếm nửa số dòng trong repo mà paper không nhắc tới.

B. Ngân sách mili-giây: một phép tính hai dòng

Cách kiểm chứng nguyên lý này bằng số học rất đơn giản: lập một ngân sách và xem con số nào phải nhường chỗ. Giả sử camera chạy 30 Hz, tức mỗi khung hình có $1000/30 \approx 33$ ms. Nếu bạn dành 4 ms cho tiền xử lý, 3 ms cho hậu xử lý và 6 ms cho phần ghép thời gian thì mô hình chỉ còn

$$33 - (4 + 3 + 6) = 20 \text{ ms},$$

và sau khi chừa biên an toàn thì thực tế là khoảng 16 ms. Công thức chỉ nói một điều: *ngân sách của mô hình là phần dư sau khi mọi khối khác đã lấy phần của chúng*, nên nó luôn nhỏ hơn con số bạn tưởng. Bây giờ đọc một paper báo cáo 45 ms trên GPU máy chủ với *batch* bằng 8: con số đó vừa vượt ngân sách hơn hai lần, vừa được đo ở chế độ batch mà bạn không bao giờ có vì bạn xử lý từng khung hình đến. Phép tính hai dòng này loại bỏ cả một họ phương pháp nhanh hơn bất kỳ bảng mAP nào. Các con số trong ví dụ là do tôi đặt ra để minh họa cách tính, không trích từ một hệ thống cụ thể.

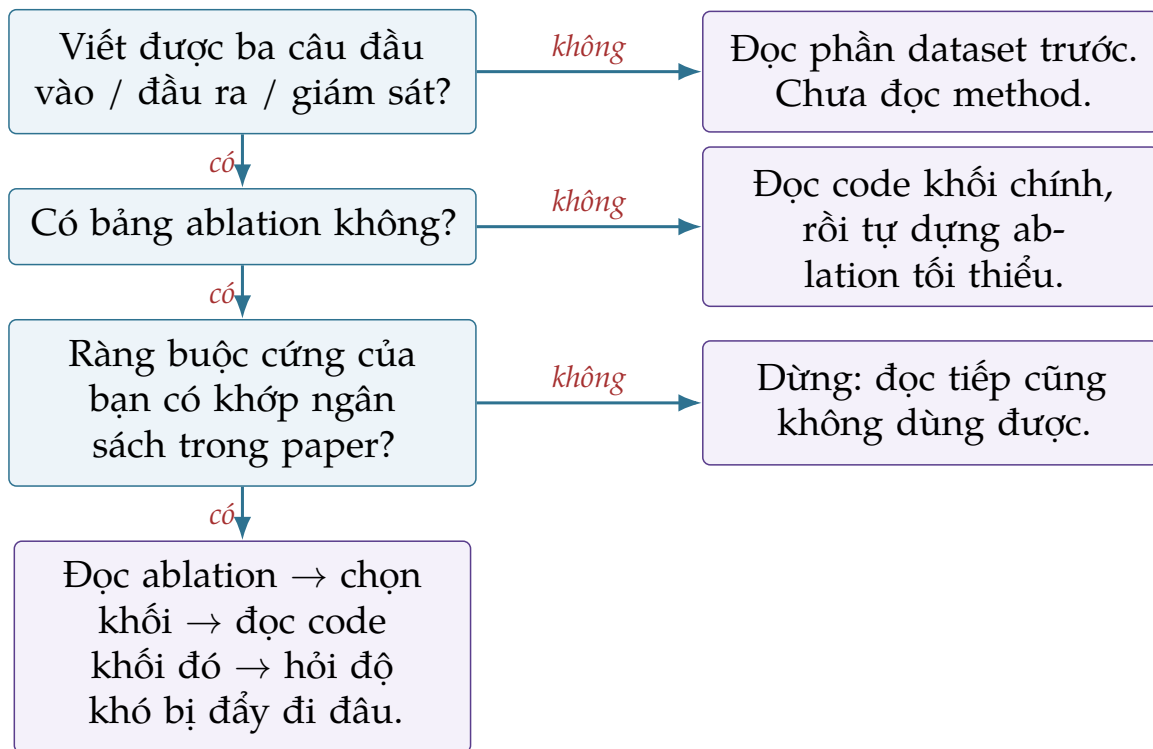
↔ **ràng buộc bởi B/11** — ngân sách mili-giây ở đây là dạng thô sơ của mô hình roofline, nơi ràng buộc thật là băng thông bộ nhớ chứ không phải số phép tính

1.5 Ba quy tắc quyết định

Bảy câu hỏi ở trên là quy trình; ba quy tắc dưới đây là các điểm dừng — những chỗ bạn được phép kết luận mà không cần đọc thêm. Giá trị của chúng nằm ở chỗ mỗi quy tắc cho phép bạn đóng một cuốn paper sớm.

1. **Chưa viết được ba câu đầu vào / đầu ra / tín hiệu giám sát?** Dừng đọc phần method, quay lại phần dataset. Đọc tiếp trong trạng thái đó chỉ nạp vào đầu các chi tiết không có chỗ để gắn.
2. **Nếu ràng buộc độ trễ siết đi năm lần thì khối nào bị thay đầu tiên?** Trả lời được câu này nghĩa là bạn đã hiểu kiến trúc theo nghĩa dùng được. Nó đòi hỏi hai thứ: biết chi phí tính toán phân bố ra sao, và biết khối nào có phương án thay thế rẻ hơn.
3. **Paper không có phần limitations?** Coi đó là tín hiệu cảnh báo về mức độ trung thực của phần còn lại, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống thật đều có chế độ hồng; im lặng về chúng là một lựa chọn biên tập.

Hình 1.2 gói ba quy tắc và mạch bảy câu hỏi thành một cây quyết định dùng được ngay khi mở một repo.



Hình 1.2. Cây quyết định khi mở một hệ thống lạ. Ba nút hỏi tương ứng ba quy tắc dừng; chỉ khi cả ba đều thoả thì việc đọc sâu mới có lãi.

1.6 Thực hành

Quy trình bảy câu hỏi chỉ trở thành phản xạ khi bạn chạy nó trên mã nguồn thật vài lần.

Repo và tài nguyên. Ba loại repo đáng mổ xẻ, theo thứ tự tăng dần độ phức tạp:

- **Tối giản, đọc hết trong một buổi** — [karpathy/minGPT](#), [lucidrains/vit-pytorch](#). Dùng để luyện tách ba tầng khi không có nhiều.
- **Production đầy đủ tầng** — [ultralytics/ultralytics](#), [open-mmlab/mmdetection](#), [facebookresearch/detectron2](#). Dùng để thấy một hệ thống thật được tổ chức ra sao.
- **Hệ thống nhiều khối ghép** — [nerfstudio-project/nerfstudio](#), [colmap/colmap](#). Dùng để thấy các đoạn nối trong Hình 1.1 mới là phần khó.

Danh sách tên mô hình dẫn đầu bảng chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; repo hạ tầng như COLMAP hay [timm](#) bền hơn nhiều, nên hãy chia thời gian theo tỉ lệ đó.

Câu hỏi kiểm tra hiểu. Trong repo bạn chọn, ranh giới giữa “bài toán”, “nguyên lý” và “chi tiết cài đặt” nằm ở đâu — chỉ đúng ba file hoặc ba hàm đánh dấu ranh giới đó. Độ khó của bài toán đã bị đẩy đi đâu, và bạn chứng minh bằng dòng code nào?

Mini project. Chọn một repo và viết *system datasheet* dài một trang: đầu vào, đầu ra, tín hiệu giám sát, ràng buộc cứng, ba giả định ngầm, ba chế độ hỏng. Ràng buộc của bài tập là không đọc paper, chỉ đọc code — đây chính là chỗ bạn phát hiện mình vẫn đang suy đoán từ tên biến thay vì từ hành vi. Vài giờ đến hai ngày.

1.7 Giới hạn và trường hợp hỏng

Quy trình này có ba chỗ dễ gây, và biết trước chúng thì bạn không tin nó quá mức.

- **Nó giả định ablation chạy đẳng hoàng** — một giả định thường sai. Ablation có thể chạy trên một cấu hình, một *seed*; trong học tăng cường người ta đã cho thấy phương sai giữa các seed đôi khi lớn hơn khoảng cách giữa các phương pháp [9]. Kinh nghiệm benchmark SLAM của bạn nói cùng một điều: một hệ đa luồng cho hai kết quả khác nhau trên cùng một bag, nên một dòng trong bảng ablation không phải một phép đo.
- **Nó tối ưu cho việc hiểu, không cho việc tái hiện.** Rất nhiều hệ thống chỉ đạt con số đã công bố nhờ những chi tiết không được coi là ý tưởng: độ dài lịch học, cách khởi tạo, một phép chuẩn hoá đặt đúng chỗ. Bạn có thể hiểu hoàn hảo một hệ thống mà vẫn không tái hiện được nó.
- **Nó không áp dụng tốt cho công trình mà đóng góp chính là dữ liệu hoặc phép đo.** Với một paper giới thiệu benchmark, bảy câu hỏi phần lớn trả về câu trả lời rỗng; khung đọc đúng là khung của chương 7 và chương 24 — tập này lấy mẫu từ đâu, chia thế nào, và metric thường cho hành vi gì.

CẠM BÃY

Cạm bẫy phổ biến nhất là nhầm “tôi đọc hiểu từng dòng code” với “tôi hiểu hệ thống”. Đọc hết code cho bạn tăng cài đặt ở độ phân giải rất cao và tăng bài toán ở độ phân giải bằng không. Dấu hiệu nhận biết: bạn mô tả được hệ thống làm gì theo trình tự thời gian, nhưng không nói được điều gì sẽ hỏng nếu ảnh đầu vào tối đi hai *stop*. Nếu rơi vào trạng thái đó, đừng đọc thêm code — hãy quay lại câu hỏi số hai.

CẦN NHỚ

Hiểu một hệ thống nghĩa là biết nó *hỏng ở đâu*, không phải biết nó chạy được gì. Quy trình gọn nhất: viết ba câu đầu vào / đầu ra / giám sát → tìm ràng buộc cứng → đọc ablation ngược → hỏi độ khó đã bị đẩy đi đâu. Nếu chưa tìm ra chỗ độ khó bị đẩy tới, bạn chưa đọc xong. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bài toán, nguyên lý, cài đặt	Ba tầng để tách một hệ thống ra trước khi đọc dòng code đầu tiên. Viết đúng một câu cho mỗi tầng thì bạn không còn nhầm một chi tiết framework với một ý tưởng cốt lõi.
••• Tín hiệu giám sát	Thứ nói cho thuật toán biết dự đoán đúng là gì: nhãn người gán, một ràng buộc vật lý, một cảm biến khác, hoặc chính dữ liệu. Biết nó đến từ đâu là biết hệ thống học được gì và không học được gì.
••• Ràng buộc cứng	Giới hạn mà tác giả không thể vượt qua: độ trễ, bộ nhớ, lượng nhân, quyền riêng tư, năng lượng. Nó loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng.
•• Vì sao ràng buộc cứng giải thích thiết kế nhiều hơn cả phần method?	Vì nó là ràng buộc loại trừ: nó xoá cả một họ phương án bất kể chúng chính xác tới đâu. Phần method chỉ xếp hạng những gì đã lọt qua.
••• Độ khó không biến mất	Nguyên lý trung tâm: không thiết kế nào làm bài toán dễ đi, nó chỉ đẩy phần khó sang dữ liệu, hậu xử lý, thời gian huấn luyện, hoặc một giả định về đầu vào. Không tìm ra chỗ bị đẩy tới nghĩa là chưa hiểu hệ thống.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Gán mẫu	Bước quyết định vị trí nào trên ảnh chịu trách nhiệm cho vật nào. Nó sinh ra đúng lúc mô hình một giai đoạn bỏ bước sinh đề xuất vùng, và là ví dụ điển hình của độ khó bị đẩy đi chỗ khác.
•• Ablation	Thí nghiệm bỏ đi từng thành phần rồi đo lại, để biết thành phần nào thực sự đóng góp. Cách đọc đúng là đọc ngược: bỏ cái gì thì hệ thống sập.
•• Đóng góp biên	Cái mà ablation thực sự đo: mức mất mát khi bỏ một khối trong khi các khối còn lại vẫn có mặt. Nó không phải giá trị tuyệt đối của khối đó.
••• Bỏ khối A không mất gì, bỏ khối B cũng không mất gì. Kết luận?	Không được kết luận cả hai vô dụng. Rất có thể chúng làm cùng một việc và che nhau; phép thử phân biệt là bỏ đồng thời cả hai, nếu hệ thống sập thì chúng dư thừa lẫn nhau chứ không thừa.
• Baseline	Phương pháp đối chứng đơn giản hơn mà phương pháp mới phải vượt qua. Nếu nó không được tinh chỉnh cẩn thận thì mọi so sánh với nó đều vô giá trị.
•• Vì sao đọc ablation trước rồi mới đọc code?	Ablation có tỉ lệ tín hiệu trên độ dài cao nhất, còn phần method thấp nhất vì nó được viết để thuyết phục người phản biện. Đọc ablation để chọn khối đáng quan tâm, rồi chỉ đọc code của khối đó.
•• Vì sao đọc limitations trước phần kết quả?	Vì đó là chỗ tác giả không có động cơ phóng đại. Bạn bước vào bảng số với một danh sách nghi vấn sẵn có, nên con số đẹp không định khung suy nghĩ của bạn.
• Paper không có mục limitations	Đọc như một tín hiệu cảnh báo về mức trung thực của phần còn lại, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống thật đều có chế độ hỏng; im lặng về chúng là một lựa chọn biên tập.
•• Sơ đồ luồng dữ liệu	Đường đi đầy đủ từ cảm biến tới quyết định cuối, có đánh dấu mọi đoạn nối. Vẽ nó ra là cách nhanh nhất thấy phần được viết paper chỉ chiếm một khối trong năm.
•• Đoạn nối	Chỗ ghép giữa hai khối: thứ tự kênh màu, chuẩn hoá, tỉ lệ toạ độ, timestamp. Phần lớn lỗi triển khai nằm ở đây chứ không nằm trong mô hình.
•• Ngân sách độ trễ	Thời gian cho mỗi khung hình, chia cho từng khối. Ngân sách của mô hình luôn là phần dư sau khi các khối khác đã lấy phần của chúng.
•• Vì sao camera 30 Hz chỉ chừa cho mô hình khoảng 16 ms?	Mỗi khung hình có 33 ms. Trừ 4 ms tiền xử lý, 3 ms hậu xử lý và 6 ms ghép thời gian còn 20 ms, rồi phải chừa biên an toàn.
•• System datasheet	Bản một trang mô tả một hệ thống: đầu vào, đầu ra, tín hiệu giám sát, ràng buộc cứng, ba giả định ngầm, ba chế độ hỏng. Viết được nó nghĩa là bạn đã đọc xong hệ thống.
•• Bạn kể được hệ thống làm gì theo trình tự nhưng không nói được nó hỏng khi nào	Bạn có tăng cài đặt ở độ phân giải rất cao và tăng bài toán bằng không. Phép thử: hỏi điều gì xảy ra nếu ảnh tối đi hai stop; trả lời không được thì quay lại ba câu đầu vào, đầu ra, giám sát.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Datasheets for datasets, model cards	Chính là “system datasheet” của chương này được chuẩn hoá thành tài liệu bắt buộc kèm mô hình và tập dữ liệu	Thông lệ ở nhiều nhóm lớn từ 2019, ít hết hạn

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
mmdetection , detectron2	Tách ba tầng bài toán / nguyên lý / cài đặt thành cấu hình, registry và module — đọc chúng là bài tập tự nhiên cho quy trình bày câu hỏi	Hạ tầng, bền
colmap/colmap	Hệ thống nhiều khối ghép, nơi các đoạn nối — hiệu chuẩn, thứ tự ảnh, ngưỡng khớp — quyết định kết quả hơn bất kỳ khối đơn lẻ nào	Hạ tầng, bền
Dòng DETR thời gian thực	Ví dụ sống của “độ khó bị đẩy đi chỗ khác”: bỏ triết phi cực đại và anchor, đổi lại chi phí hội tụ và cả một dòng nghiên cứu về thiết kế query	Tính tới 9/2026

MINI PROJECT SLAM · System datasheet cho ORB-SLAM3, chỉ đọc code · 1–2 ngày

Mục tiêu	biến quy trình bày câu hỏi thành một sản phẩm kiểm tra được, trên chính hệ thống bạn chạy hằng ngày. Bài tập sẽ lộ ra những chỗ bạn tưởng mình hiểu mà thực ra chỉ đang suy đoán từ tên biến.
Dữ liệu	mã nguồn ORB-SLAM3 [1] và một cấu hình EuRoC đang chạy được. Ràng buộc của bài tập: <i>không mở paper</i> , chỉ đọc code và file cấu hình.
Các bước	viết một trang gồm đúng sáu phần — (1) đầu vào, đầu ra, tín hiệu giám sát; (2) ràng buộc cứng đọc được từ code, ví dụ số đặc trưng tối đa, số mức kim tự tháp, ngưỡng số điểm khớp để coi là bám được; (3) sơ đồ luồng dữ liệu từ ảnh tới pose, đánh dấu mọi đoạn nối; (4) ba giả định ngầm, mỗi giả định kèm dòng code chứng minh; (5) ba chế độ hỏng suy ra từ ba giả định đó; (6) độ khó đã bị đẩy đi đâu.
Xong khi	bạn trả lời được bằng văn bản câu “nếu ràng buộc độ trễ siết đi năm lần thì khối nào bị thay đầu tiên”. Câu trả lời phải kèm phần trăm thời gian của từng khối, đo bằng bộ đếm bạn tự chèn chứ không bằng cảm giác. Kiểm tra chéo: ba chế độ hỏng bạn nêu phải tái hiện được ít nhất một cái trên dữ liệu thật (ví dụ làm mờ ảnh đầu vào hoặc giảm số đặc trưng) và quan sát đúng triệu chứng bạn đã dự đoán.
Nối với	B/05 dùng lại datasheet này khi thiết kế bài toán học máy đầu tiên gắn vào hệ; B/11 thay ước lượng phần trăm thời gian bằng mô hình roofline; E/24 cho cách đọc con số đo được khi harness có thành phần ngẫu nhiên.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai người đọc theo hai cách phân loại khác nhau: người thứ nhất theo đặc điểm bề mặt (tên module, tên tầng), người thứ hai theo cấu trúc sâu (bài toán, giả định, đánh đổi). Chỉ bản đồ giả định mới sinh ra dự đoán về triệu chứng.
- A2. Vì phần method có tỉ lệ tín hiệu trên độ dài thấp nhất — nó được viết để thuyết phục người phân biện — trong khi ablation và limitations nói thẳng phần nào đóng góp và phần nào hỏng. Đọc ablation trước để chọn điểm vào, rồi đọc code của đúng phần đó.
- A3. Vì ràng buộc cứng là ràng buộc loại trừ: nó xoá cả một họ phương án bất kể chúng chính xác tới đâu, còn bảng kết quả chỉ xếp hạng trong số những phương án đã lọt qua. Thêm nữa, 45 ms đó thường được đo ở chế độ batch mà hệ thống thời gian thực của bạn không bao giờ có.
- A4. Không. Ablation đo đóng góp biên khi các khối còn lại đã có mặt, nên hai khối dư thừa lẫn nhau sẽ cùng cho ra “bỏ đi không mất gì” trong khi bỏ cả hai thì hệ thống sập. Đây là lý do ablation không thay được việc đọc code.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn ngồi ăn ở một quán cơm nhỏ và muốn hiểu vì sao thực đơn của họ đúng như vậy. Bạn có thể hỏi chủ quán, và được nghe một câu chuyện đẹp về công thức gia truyền. Bạn có

thể xin vào bếp xem họ nấu, và thấy chính xác những gì đang xảy ra nhưng vẫn không hiểu vì sao. Nhưng nếu bạn đo cái bếp — chỉ có hai bếp lửa, một tủ lạnh nhỏ, và khách phải có đồ ăn trong tám phút — thì gần như cả thực đơn tự giải thích. Món nào cần hầm ba tiếng thì phải hầm sẵn từ sáng: công sức không mất đi, nó chỉ chuyển sang lúc khác.

Đọc một hệ thống máy tính lạ cũng vậy. Bài toán gốc là bạn có vài trăm nghìn dòng mã cùng một bài báo dài, còn thời gian thì chỉ một buổi. Mẹo là đừng bắt đầu từ ý tưởng, hãy bắt đầu từ những giới hạn mà người làm ra nó không thể vượt qua: máy phải trả lời trong bao nhiêu phần nghìn giây, có sẵn bao nhiêu dữ liệu đã được đánh dấu, bộ nhớ bao nhiêu. Những giới hạn ấy gạt bỏ phần lớn cách làm khác trước khi chuyện hay dở kịp được bàn tới.

Cái giá phải trả là bạn buộc phải đọc kỹ đúng phần trông nhàm chán nhất, và để dành phần hấp dẫn nhất tới cuối.

Chỗ không rút gọn được: chuyện “bỏ bộ phận nào ra thì máy vẫn chạy, nên bộ phận đó vô dụng” thì không rút gọn được mà vẫn đúng. Nếu hai bộ phận cùng làm một việc, bỏ riêng từng cái đều không thấy mất gì, mà bỏ cả hai thì máy đứng. Mọi cách nói ngắn hơn điều này đều dẫn tới kết luận sai.

Một câu: Muốn hiểu một cái máy lạ, hãy đo cái hộp mà người ta buộc phải nhét nó vào, đừng nghe câu chuyện về nó.

1.8 Kết nối

Chương này là tiền đề của cả Phần B; mỗi chương sau là một lát cắt chi tiết của một trong bảy câu hỏi.

- **Câu 2 (đầu vào / đầu ra / giám sát)** → B/05-thiet-ke-bai-toan: ở đó việc chọn đơn vị phân tích và dạng đầu ra khoá chặt mọi thứ phía sau.
- **Câu 3 (ràng buộc cứng)** → B/11-gioi-han-tinh-toan: ngân sách mili-giây trong ví dụ trên được thay bằng mô hình *roofline* nghiêm túc [10].
- **Câu 4 (độ khó bị đẩy đi đâu)** → B/04-nen-tang-hoc-tu-du-lieu: ở đó nó mang tên chính thức là trục đánh đổi giả định ↔ dữ liệu.
- **Câu 5 (ablation đáng tin không)** → chương 24 ở Phần E.
- **Hai khối đầu của Hình 1.1** → A/02-vat-ly-tao-anh, tiền đề vật lý của chúng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ràng buộc cứng loại bỏ nhiều phương án thiết kế hơn cả phần method cộng lại, và điều đó gợi ý gì về thứ tự đọc một paper?
2. Trong một repo bạn đã đọc, độ khó đã bị đẩy đi đâu — và bạn kiểm chứng bằng bằng chứng nào trong code?
3. Bảng ablation che giấu điều gì mà việc đọc code lộ ra, và trong tình huống cụ thể nào thì nó kết luận sai?
4. Vì sao nên đọc phần limitations trước phần kết quả, chứ không phải sau?
5. Nếu ngân sách độ trễ bị siết từ 33 ms xuống 8 ms mỗi khung hình, khối nào trong Hình 1.1 bị thay đầu tiên, và vì sao không phải khối mô hình?
6. Quy trình bảy câu hỏi hổng ở loại công trình nào, và khung đọc nào thay thế nó ở đó?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.